



**QUEEN'S
UNIVERSITY
BELFAST**

On the Virtualization of CUDA Based GPU Remoting on ARM and X86 Machines in the GVirtuS Framework

Montella, R., Giunta, G., Laccetti, G., Lapegna, M., Palmieri, C., Ferraro, C., Pelliccia, V., Hong, C.-H., Spence, I., & Nikolopoulos, D. (2017). On the Virtualization of CUDA Based GPU Remoting on ARM and X86 Machines in the GVirtuS Framework. *International Journal of Parallel Programming*, 45(5), 1142-1163.
<https://doi.org/10.1007/s10766-016-0462-1>

Published in:
International Journal of Parallel Programming

Document Version:
Peer reviewed version

Queen's University Belfast - Research Portal:
[Link to publication record in Queen's University Belfast Research Portal](#)

Publisher rights
© 2016 Springer Verlag.
The final publication is available at Springer via <http://dx.doi.org/10.1007/s10766-016-0462-1>

General rights
Copyright for the publications made accessible via the Queen's University Belfast Research Portal is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy
The Research Portal is Queen's institutional repository that provides access to Queen's research output. Every effort has been made to ensure that content in the Research Portal does not infringe any person's rights, or applicable UK laws. If you discover content in the Research Portal that you believe breaches copyright or violates any law, please contact openaccess@qub.ac.uk.

Open Access
This research has been made openly available by Queen's academics and its Open Research team. We would love to hear how access to this research benefits you. – Share your feedback with us: <http://go.qub.ac.uk/oa-feedback>

On the virtualization of CUDA based GPU remoting on ARM and X86 machines in the GVirtuS framework

**Raffaele Montella · Giulio Giunta ·
Giuliano Laccetti · Marco Lapegna ·
Carlo Palmieri · Carmine Ferraro ·
Valentina Pelliccia · Cheol-Ho Hong ·
Ivor Spence · Dimitrios S. Nikolopoulos**

Received: date / Accepted: date

Raffaele Montella
University of Napoli Parthenope
E-mail: raffaele.montella@uniparthenope.it

Giulio Giunta
University of Napoli Parthenope
E-mail: giulio.giunta@uniparthenope.it

Giuliano Laccetti
University of Napoli Federico II
E-mail: giuliano.laccetti@unina.it

Marco Lapegna
University of Napoli Federico II
E-mail: marco.lapegna@unina.it

Carlo Palmieri
University of Napoli Parthenope
E-mail: carlo.palmieri@uniparthenope.it

Carmine Ferraro
University of Napoli Parthenope
E-mail: carmine.ferraro@uniparthenope.it

Valentina Pelliccia
University of Napoli Parthenope
E-mail: valentina.pelliccia@uniparthenope.it

Cheol-Ho Hong
Queen's University of Belfast
E-mail: c.hong@qub.ac.uk

Ivor Spence
Queen's University of Belfast
E-mail: i.spense@qub.ac.uk

Dimitrios S. Nikolopoulos
Queen's University of Belfast
E-mail: d.nikolopoulos@qub.ac.uk

Abstract The astonishing development of diverse and different hardware platforms is twofold: on one side, the challenge for the exascale performance for big data processing and management; on the other side, the mobile and embedded devices for data collection and human machine interaction. This drove to a highly hierarchical evolution of programming models. GVirtuS is the general virtualization system developed in 2009 and firstly introduced in 2010 enabling a completely transparent layer among GPUs and VMs. This paper shows the latest achievements and developments of GVirtuS, now supporting CUDA 6.5, memory management and scheduling. Thanks to the new and improved remoting capabilities, GVirtuS now enables GPU sharing among physical and virtual machines based on x86 and ARM CPUs on local workstations, computing clusters and distributed cloud appliances.

Keywords GPGPU · HPC · ARM · Cloud · Virtualization

1 Introduction

In the challenge for the enormous benefits of exascale applications, the Top500 ranking and its greener counterpart, the Green500 list, an impressive improvement is shown in the performance-power ratio of large-scale high performance computing (HPC) facilities over the last five years. Furthermore, a trend clearly visible in these two lists is the adoption of hardware accelerators to obtain unprecedented levels of raw performance with reasonable energy costs, which hints that future Exaflop systems will most likely leverage some sort of specialized hardware [40].

The virtualization currently provided by popular open source hypervisors (XEN, KVM, Virtual Box) does not allow software based transparent use of accelerators as CUDA based GPUs. VMWare and XEN support GPU on the basis of hardware virtualization provided natively by NVIDIA GRID devices instead [16].

High Performance Internet of Things (HPIoT) and High Performance Cloud Computing (HPCC) are typical examples of highly heterogeneous computing systems, where different devices and computing units coexist in the same software environment[8]. They can be described as highly parallel internet-based models providing virtualized and standard resources as a service over the Internet.

In this paper the evolution of our GVirtuS (Generic Virtualization Service) enabling transparent GPGPU virtualization[12] and remoting[13] for low-power processors for, but not limited to, the acceleration of scientific applications is presented [28]. In the latest GVirtuS incarnation the architecture independence was enforced, in order to make it work with both CUDA and OpenCL on Intel and ARM architecture, as well as with a clear roadmap heading to Power architectures compatibility. The rest of the paper is organized in the following way: section 2 is a brief technical introduction about GVirtuS, its design, architecture and implementation; section 3 is a detailed description of the GVirtuS new features and how the heterogeneous architectures support

has been enabled; section 4 is about the experiment setup for different scenarios; section 5 shows the evaluation results; in section 6 the current version of GVirtuS with other notable related works are compared and contrasted; finally, section 7 is about conclusions and future directions of this promising research.

2 GVirtuS: a tool to virtualize heterogeneous architectures

GVirtuS is a generic virtualization framework for virtualization solutions based on a split-driver model [1]. GVirtuS offers virtualization support for generic libraries such as accelerator libraries (CUDA, OpenCL), with the advantage of independence from all involved technologies: hypervisor, communicator and target of virtualization. This feature is possible thanks to the plug-in design of the framework, enabling the choice of different communicator or different stub-libraries mocking the virtualization target. GVirtuS is transparent for developers: no changes are required in the software source code to virtualize and execute and there is no need to recompile an already compiled executable.

Low-power processors as ARM or Intel technologies are employed in diverse and different environments for the resolution of highly complex scientific problems, because their low cost and reduced cooling needs. On the other hand, the use of ARM CPUs in HPC infrastructures is a cutting edge technology, but, apparently, not ready for the prime time. At present, most scientific applications are too demanding of high performance to run on the current generation of ARM CPUs, even when integrated with GPUs. To accelerate the use of ARM in science production, remoting capabilities in GVirtuS have been improved in order to share high-end GPU devices hosted on x86 machines with low power/low cost ARM based computing clusters. This implies important challenging issues from the architectural point of view, partially mitigated by the GVirtuS modular design. Some requirements had to be set firmly in order to make it possible, as the use of an ARM CPUs with endianness and word length coherent to the x86 ones.

2.1 Architecture, design and implementation

GVirtuS strictly depends on CUDA APIs version because the nature of the transparent virtualization and remoting. In this paper we show our results in GVirtuS development relying on the the CUDA 6.5 APIs. The use of this version is motivated by the following issues:

- After the release of the CUDA 3.0 APIs, the library design no longer fits the same split-driver approach used by GVirtuS and other similar products;
- The CUDA 6.5 APIs unchain the CUDA power on tiny low power ARM architecture: CUDA applications can be compiled directly on the ARM board if ad hoc libraries available from NVIDIA are installed;

- CUDA is strictly proprietary and not open source, making the use of a virtualization/remoting layer non trivial. The GVirtuS development is framed in a wider big picture where the target application requirements are CUDA 6.5 compliant.

Since the first public release, the GVirtuS development has been characterized by two main goals: providing a fully transparent virtualization/remoting solution; reducing the overhead of virtualization and remoting to make the performance of the virtualized solution as close as possible to the bare metal execution.

The front-end/back-end communication is abstracted by the Communication interface concretely implemented by each communicator component. This issue is critical, especially when the virtualized resources need to be thread-safe, as in case of GPUs providing CUDA support. The methods implemented in this class support request preparation, input parameters management, request execution, error checking and output data recovery. The Handler class provides the base functionalities for each stub function management. The back-end is executed on the host machine behaving as a server component running as a user with enough privileges to interact with the CUDA driver. The back-end accepts a new connection spawning a new process to serve the front-end requests. The CUDA enabled application running on the virtual or remote machine requests GPGPU resources to the virtualized device using the stub-library. Each function in the stub-library follows these steps:

- Obtains a reference to the single Frontend instance;
- Uses Frontend class methods for setting the parameters;
- Invokes the Frontend handler method specifying the remote procedure name;
- Checks the remote procedure call results and handles output data.

In order to implement the NVIDIA CUDA stack split-driver using GVirtuS, a developer has to subclass from Frontend, Backend and Handler classes. For CUDA runtime virtualization the handler is implemented as a collection of functions and a jump table for a specified service. As in GVirtuS predecessor gVirtuS, in the case of CUDA runtime virtualization, the front-end has been implemented as a dynamic library based on the interface of the original libcudart.so library. Beginning with the second generation of GVirtuS component, the virtualization is focused on CUDA, but not limited to it. Thanks to the GVirtuS modularity and technology/architecture independence, the plug-ins for openCL and, partially, openGL have been developed. The CUDA driver implementation is similar to the CUDA runtime, except for the low-level ELF binary management for CUDA kernels. A slightly different strategy has been used for openCL and openGL support: the openCL library provided by NVIDIA is a custom implementation of a public specification.

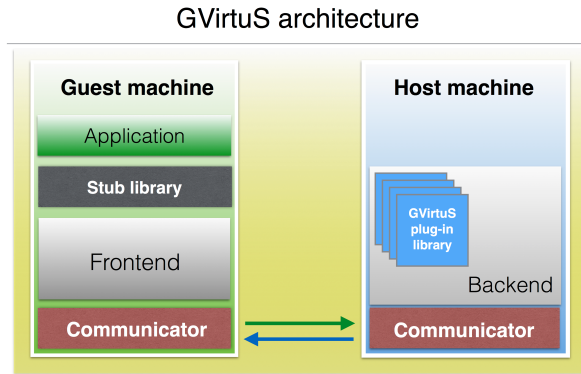


Fig. 1 The GVirtuS approach to the split-driver model.

2.2 The front-end

The front-end leverages on the driver's APIs supported by the platform running on a virtual machine instance or on a remote physical machine and is implemented as a stub-library. A stub-library is a virtualization of the real APIs library on the client operating system where the application is launched (typically a virtual machine or a physical one without GPU support). The stub-library implements the functionality of the host machine (GPU capable) on the guest machine. The role of the front-end is to intercept calls to the functions of APIs supported, transfer to the back-end the parameters passed to functions through the use of the selected communicator and wait for the execution result from the back-end. This result is made possible by the stub-library that provides the driver APIs abstraction to the guest application. When a client application calls a function, the stub-library intercepts the call and packs the serialized parameters in a buffer data structure. The front-end sends to the back-end the serialized buffer and the name of the function called through the communicator waiting for the response. For each method of the APIs there is a corresponding method for the management and the execution in the front-end.

2.3 The back-end

The back-end is the main component of the GVirtuS framework and runs on the host machine (GPU capable). The back-end daemon runs on the host operating system in the user or superuser space, depending on the specifics of applications and security policies waiting for an incoming connection from the front-end. The daemon implements the back-end functionality dealing with the physical device driver and performing the host-side virtualization. When it receives a request, the back-end creates a new process and loads the plug-in needed for the requested function execution. After this operation, the back-end

is ready for a new request from another guest machine. The new process reads the name of the API called, calls the associated method for managing the API required, allocates the space for the parameters of the method required and inserts the value from the parameters passed in the buffer from the front-end. The back-end calls the real API on the host machine through direct access to the driver of the physical device and saves the result in another buffer. Finally, the buffer result is passed to the front-end of the guest machine through the Communicator. To each method of the APIs corresponds a method for the management and the execution in the back-end.

2.4 The communicator

The communicator is an important component of the GVirtuS framework connecting the front-end guest machine to the back-end host machine. The communicator is independent of hypervisor and virtualized technology. The communicators have strict high-performance requirements because they are used in system-critical components split-driver model compliant. The communicator provides a secure, high-performance, direct communication mechanism between the two sides of virtualization or remoting. The choice of the communicator depends on the physical machine connectivity, in both host and guest machines, because it influences the virtualization performance. GVirtuS provides several communicator implementations, including the TCP/IP communicator. The TCP/IP communicator is used for supporting virtualized and distributed resources. In this way, a virtual machine running on a local host can access a virtual resource physically connected to a remote host in a transparent way. In practice, the Communicator serializes the buffer structure and implements the transmission between host and guest.

3 Remoting and novelty introduced features

In order to fit the GPGPU/x86/ARM application into our generic virtualization system, the back-end on the x86 machine directly connected to the GPU based accelerator device and the front-end on the ARM board(s) using the GVirtuS tcp/ip based communicator have been mapped. GVirtuS as NVIDIA CUDA remoting and virtualization tool achieves good results in terms of performances and system transparency.

CUDA applications are executed on the ARM board through the GVirtuS front-end. Thanks to the GVirtuS architecture, the front-end is the only component needed on the guest side. This component acts as a transparent virtualization tool giving to a simple and inexpensive ARM board the illusion to be directly connected to one or more high-end CUDA enabled GPGPU devices.

The diagram (Figure 2) shows the computing architecture (ARM, x86_64) and the acceleration model (CUDA, OpenCL) independence. GVirtuS currently supports a growing subset of NVIDIA CUDA features. Thanks to the

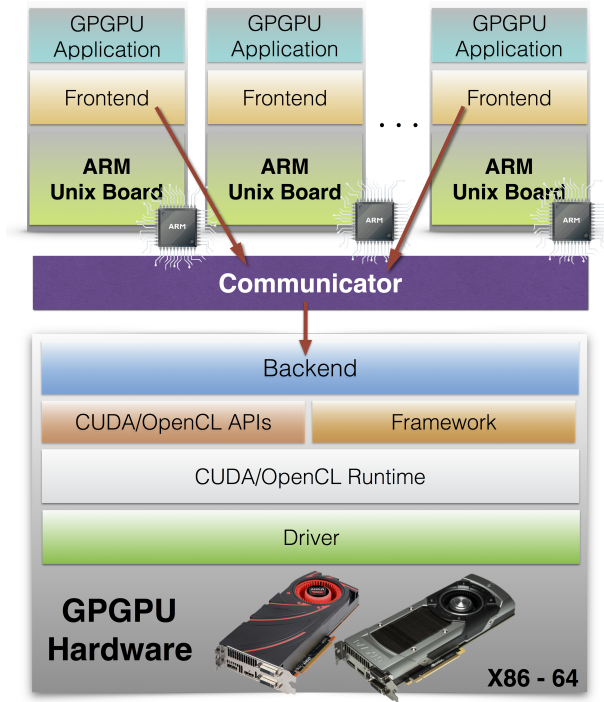


Fig. 2 The GVirtuS architecture.

GVirtuS modular design, some new features have been developed, such as the GPU scheduling.

3.1 Unified Virtual Addressing (UVA) management

The Unified Memory, introduced with CUDA version 6.x, simplifies the programming model by enabling applications to access CPU and GPU memory without the need to manually copy data from one to the other, and makes it easier to add support for GPU acceleration in a wide range of programming languages. When there is no distinction between a host and device pointer, CUDA runtime can identify where the data are stored and the correct value of the pointer. Essentially, in a Unified Virtual Address any space allocated through the `cudaMalloc`, `cudaMallocManaged`, `cudaHostAlloc` or `cudaMallocHost` functions is mapped in a single unified space. As a consequence, for example, the direction of the copy in a `cudaMemcpy` function becomes obsolete so it is replaced by `cudaMemcpyDefault`. To support these features in GVirtuS, maps and lists have been used (Figure 3).

Anytime a call to a function is made from the `cudaMalloc` family, the result pointer is stored in a list, so the nature of the pointer can be easily identified. When a call to `cudaMemcpy` is performed, the front-end can correctly identify

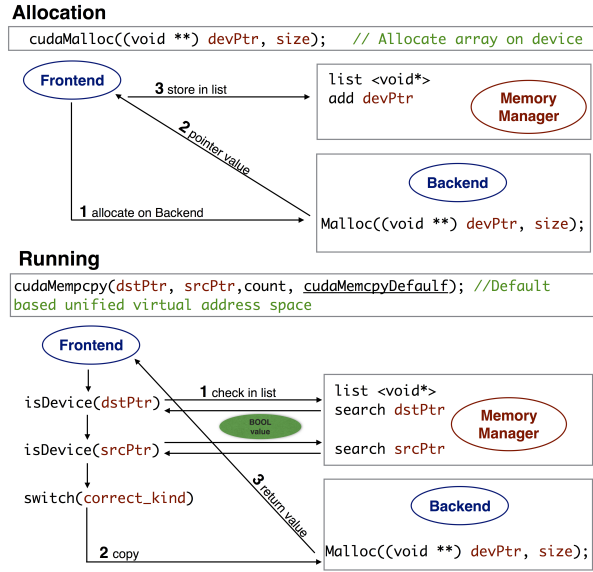


Fig. 3 Automatic memory management.

the direction even when the `cudaMemcpyDefault` flag is selected. Direction mismatch is also avoidable, but this feature is not provided by CUDA, so it has not been used for this project. The nature of the pointers is determined by querying the list where the device pointers are stored. To support the UVA any time a managed pointer is allocated, this is stored in a map along with its size and a host pointer allocated through `malloc` function from `glibc`. When a managed pointer is involved in the execution, GVirtuS runtime takes care that data are passed to the back-end and stored on the device. Moreover, GVirtuS runtime takes care that the processed data are available on the front-end after the execution. When a managed pointer has to be used, the GVirtuS runtime searches for a match on the pointer map ensuring the coherence between the two virtualization/remoting address spaces. The memory pinned by the managed pointer is copied from the front-end to the back-end bounded with the valid device pointer. Finally, the GVirtuS runtime pushes in a stack the value of the host pointer.

All the pointers present in the stack after the computation are transferred from the back-end to the front-end, so that the processed data are available on the front-end side. At this stage, no significative overhead is introduced by the identification process. Nevertheless, in the UVA case a significant overhead is introduced, because any pointer involved in the calculation must be enforced by coherence in both directions (Figure 4).

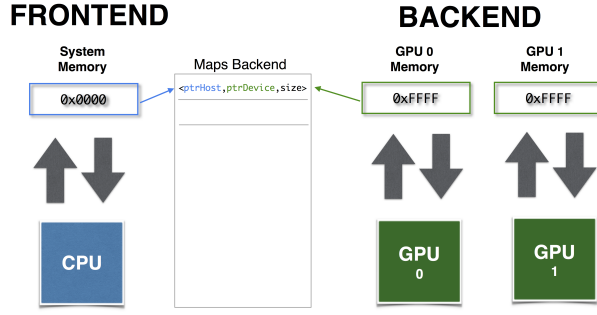


Fig. 4 Unified Virtual Address with back-end maps.

3.2 GPU Scheduling

The GPU scheduler of GVirtuS enables fair and efficient use of virtualized GPUs among multiple cloud users. The GPU scheduler multiplexes back-end processes spawned by the GVirtuS Backend driver; GVirtuS creates a back-end process whenever a connection between the split-drivers is established, and terminates it after the connection between them is closed. The scheduler maintains a run queue to accommodate runnable back-end processes and selects one of them to execute according to its fairness policy. It then gives a token to the chosen process in order to allow the process to exclusively access GPU devices during its time slice.

As a fairness policy, the GPU scheduler adopts the Credit scheduling policy, which is a proportional fair-share algorithm employed in the Xen hypervisor as a CPU scheduler. In this scheduling policy, the global credit accounting function periodically (30 ms) assigns a certain amount of credits to each back-end process in proportion to the GPU weight. The accounting function then decides the priority of each process based on the remaining credit amount. Similarly to the Xen hypervisor, the GPU scheduler maintains two priorities: UNDER and OVER. If the credit value of a back-end process is positive, its priority is set to UNDER. Otherwise, the priority becomes OVER. The accounting function then sorts the back-end processes into priority order (UNDER and OVER) in the run queue; for simplicity and fast sorting speed, the scheduler does not sort them based on the credit amount.

The scheduler selects the next back-end process to run in the head of the run queue at every scheduling instance; it implements the $O(1)$ scheduling concept that can select the next process within a fixed amount of time. While the chosen process is using GPU devices, the credit value of the process is decreased at a fixed rate. After the time slice, the back-end process is put at the tail of its priority list. Before the global credit accounting function executes, its priority is maintained regardless of its current credit amount in order to reduce the frequency of sorting. This whole procedure reflects the

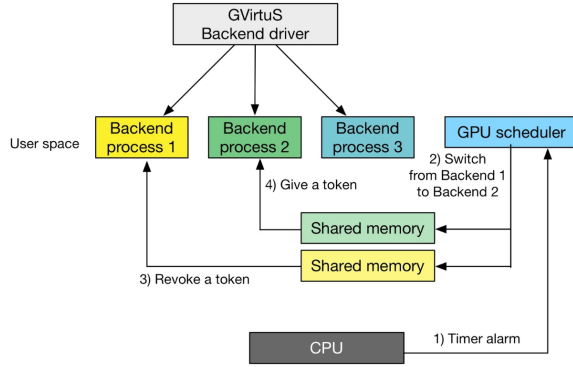


Fig. 5 Procedure of GPU back-end process switching from back-end process 1 to 2.

motivation of Credit scheduling, which focuses on efficient scheduling decision and simple implementation.

As depicted in Figure 5, the GPU scheduler is placed in the user space of the host OS rather than in the kernel space in order to communicate with back-end processes more efficiently. It utilizes POSIX shared memory and real time signal mechanisms for synchronous and asynchronous communication respectively. When a back-end process is created, it notifies the GPU scheduler of its process ID by a signal. The GPU scheduler then inserts the process in the GPU run queue. When a timer alarm event is sent to the GPU scheduler, the scheduler decides the next back-end process to execute based on the Credit scheduling policy. The scheduler then revokes a token from the previous process and delivers it to the next process via the shared memory. In our implementation, the time slice of a running back-end process is configured to 6 ms, which can be adjusted by the administrator.

4 Scenarios and prototypal applications

Designing a test plan for the application scenario described in this paper is a complex issue because, while many legacy CUDA enabled applications and widely accepted performance test cases are available for the x86_64 architecture, the same is not true for ARM. So we had to face the lack of standard testing guidelines for CUDA and ARMs due to the weak available support for this technology, relatively new for CUDA environment. From the hardware point of view, our test setup involves a Maxwell based development workstation and a cluster built of 3 ARM based high-end single board computers (SBCs).

In order to test performance for x86_64/x86_64/GPU two different evaluation software have been used: CUDASW++ [23] and SRAD [35]. The first is a bioinformatics software for Smith-Waterman protein database searches, while the latter is a diffusion method for ultrasonic and radar imaging applications

based on partial differential equations. Both applications take advantage of the massively parallel CUDA architecture of NVIDIA.

ARM/x86/GPU performance tests have been produced developing an ad hoc MPI Matrix Multiplication software [39] enabling the GVirtuS behaviour investigation setting up a scenario where a x86 machine is used as an accelerator node of a high-end ARM based cluster. The Matrix Multiplication software used is a matrix-matrix multiply routine (GEMM, General Matrix to Matrix Multiplication) achieving better performance if compared with other usual implementations. This routine uses a LU, QR, and Cholesky factorizations gaining up to 80-90% of the peak GEEM rate thanks to the strict adherence to modern GPUs programming guidelines.

4.1 The development workstation

The performance test system has been built on top of the Ubuntu 14.04 Linux operating system, the NVIDIA CUDA Driver, and the SDK/Toolkit version 6.5 hosted on a workstation equipped by an i7-940@2.93 GHz 12Gb RAM. The GPU subsystem is enforced by two NVIDIA GeForce Titan X 12Gb RAM powered by the Maxwell chipset and summing up 3072 CUDA cores.

4.2 The cluster based on high-end ARM Single Board Computer

In order to face with a real next generation high performance computing scenario, an experimental cluster made by 3 NVIDIA Jetson TK1 computing nodes has been set up, connected by a dedicated Gigabit Ethernet network to the developing workstation mimicking an accelerator server. Each computing node relies on 4-PLUS-1 Cortex A15 r3 CPU architecture, that delivers higher performance and is more power efficient than the previous generation, and a Kepler GPU architecture that utilizes 192 CUDA cores to deliver advanced graphics capabilities, GPU computing with NVIDIA CUDA 6.x support, breakthrough power efficiency and performance for the next generation of gaming and GPU-accelerated computing applications.

4.3 Smith-Waterman sequence alignment

The Smith-Waterman algorithm has been available for more than 25 years. It is based on a dynamic programming approach that explores all the possible alignments between two sequences; as a result it returns the optimal local alignment. Unfortunately, the computational cost is very high, requiring a number of operations proportional to the product of two-sequence length. Furthermore, the exponential growth of protein and DNA databases makes the Smith-Waterman algorithm unrealistic for searching similarities in large sets of sequences [21]. The alignment of two sequences is based on the computation of an alignment matrix. The number of its columns and rows is given by the

number of the residues in the query and database sequences respectively. The computation is based on a substitution matrix and on a gap-penalty function. The CUDASW++ [22] has been used as evaluation software. It is a publically available open source software for Smith-Waterman protein database searches on Graphics Processing Units with CUDA. This software has been added to the NVIDIA Tesla Bio Workbench.

4.4 Rodinia performance study application

The Rodinia software suite is widely accepted by the GPGPU scholars as a test of CUDA performances and capabilities. It uses the Berkeleys dwarf taxonomy to choose the applications developed using CUDA and OpenMP. Each dwarf represents a set of algorithms with similar computation and data movement. Even though programs representing a particular dwarf may have varying characteristics, they share strong underlying patterns. The dwarves are defined at a high level of abstraction to allow reasoning about the program behaviors [36]. The Rodinia software suite focuses on Structured Grid, Unstructured Grid, Combinational Logic, Dynamic Programming, Fast Fourier Transform (FFT), N-Body, Monte Carlo and Dense Linear Algebra dwarves. It targets on GPUs and multicore CPUs as a starting point in developing a broader treatment of heterogeneous computing. Rodinia benchmark suite enables users to evaluate heterogeneous systems including both accelerators, such as GPUs, and multicore CPUs. The parallel computing on GPGPU application chosen to test GVirtuS is Speckle Reducing Anisotropic Diffusion (SRAD) [38]. SRAD is a diffusion method for ultrasonic and radar imaging applications based on partial differential equations (PDEs). It is used to remove locally correlated noise, known as speckles, without destroying important image features. SRAD consists of several pieces of work: image extraction, continuous iterations over the image (preparation, reduction, statistics, computation 1 and computation 2) and image compression.

4.5 Matrix multiplication

Our implementation of matrix multiplication takes advantage of shared memory already used to evaluate the performances ARM CUDA enabled software offloaded on remoted GPUs [24].

In this implementation each task (MPI process or thread) is responsible for computing `number_of_rows/number_of_task` rows of the matrix `C` (Algorithm 1). Every block of CUDA thread is responsible for the computing of one square sub-matrix `Csub` of `C` and each thread within the block is responsible for computing one element of `Csub`. `Csub` is equal to the product of two rectangular matrices: the sub-matrix of `A` of dimension `(A.width, block_size)` that has the same row indices as `Csub`, and the sub-matrix of `B` of dimension `(block_size, A.width)` that has the same column indices as `Csub`. In order to

fit into the device resources, these two rectangular matrices are divided into as many square matrices of dimension `block_size` as necessary and `Csub` is computed as the sum of the products of these square matrices [3]. In order to easily verify the correct execution of the code the software performs:

$$RAND(n \times n) * EYE(n \times n) = RAND(n \times n) \quad (1)$$

The choice of this strategy comes from the easy scalability and evaluation, and because it does not need synchronization mechanism to avoid race condition, this comes from the spawn of the data amongst the tasks. We propose two implementations of this test: one for MPI process and one for POSIX thread. The main process (Rank 0 in MPI) takes care of distributing the data amongst the workers and collecting them after the computing process is ended.

Algorithm 1 MatrixMul MPI/CUDA

```

1: procedure MAINTASK
2:   for  $i \leftarrow 1, \text{num\_of\_tasks}$  do
3:      $alocal \leftarrow a[\text{offset} * \text{num\_rows\_a}]$ 
4:      $clocal \leftarrow c[\text{offset} * \text{num\_rows\_b}]$ 
5:     SEND_TO_WORKER( $alocal$ )
6:     SEND_TO_WORKER( $b$ )
7:     SEND_TO_WORKER( $clocal$ )
8:   end for
9:   for  $i \leftarrow 1, \text{num\_of\_tasks}$  do
10:    COLLECT_FROM_WORKER( $i$ )
11:   end for
12: end procedure
13: procedure WORKERTASK
14:   for  $a \leftarrow 1, \text{num\_of\_el\_a}$  do
15:     for  $k \leftarrow 1, \text{num\_of\_block}$  do
16:        $Csub \leftarrow \text{CALCULATE\_C\_SUBMATRIX}(k)$ 
17:     end for
18:      $C \leftarrow \text{COLLOCATE\_CSUB}(Csub)$ 
19:   end for
20: end procedure

```

Writing a basic dense matrix-matrix multiplication kernel is a fairly simple exercise (see the CUDA Programming Guide for details). Achieving this high level of performance, on the other hand, requires more careful optimization. Volkov and Demmel used a block algorithm similar to those used for vector computers, using GPU registers and per-block shared memory to store the data blocks [39]. As the GPU has an unusually large register file, registers can be used as the primary scratch space for the computation. Furthermore, assigning small blocks of elements to each thread, rather than a single element to each thread, boosts efficiency much as strip-mining boosts efficiency on vector machines. Finally, the non-blocking nature of loads on the GPU makes it possible to do software prefetching, which is useful for hiding memory latency [11].

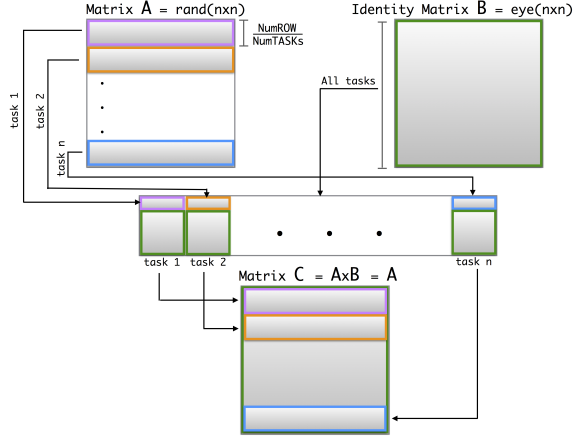


Fig. 6 Implementation of Matrix Multiplication multitask.

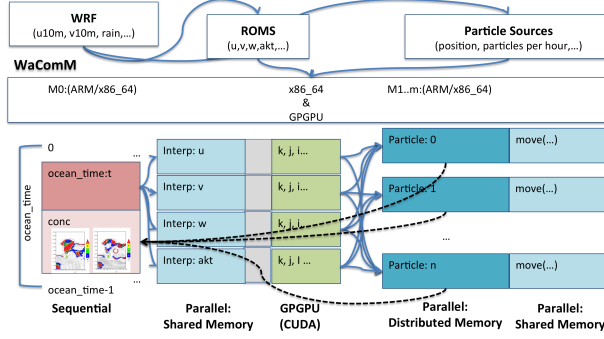


Fig. 7 The WaComM hybrid parallel implementation.

4.6 Experiment design for a real world problem

WaComM (Water quality Community Model) is a coastal area decisionmaking tool for mussel farms food quality assessment and prediction. It is based on eulerian/lagrangian methods. WaComM is numerically coupled with marine dynamics models[10] in an offline fashion. WaComM has been developed and tested in X86_64 multicore environments. Due to the intensive demanding computations, the porting to a hierarchically parallel architecture has been designed and partially already implemented. In this scenario we refactored the model code in order to implement distributed memory/shared memory/GPGPU hierarchical parallelisation (Figure 7).

The use of GVirtuS in order to take advantage of a massive ARM based HPC system with few high-end CUDA equipped accelerator nodes could be the killing application targeting the reduction of total cost of ownership (procurement, powering, cooling) in an application field, the continuous operational real-time environmental modelling, where the on-premises and on-cloud solu-

tions are economically borderline options. A forthcoming paper will discuss about the WaComM architecture, its implementation and the performance assessment in a GVirtuS based, mixed ARM/X86_64/GPGPU environment.

5 Results

From the wall clock time point of view, the execution of an application interacting with a remoted GPU will sustain lower performance than the same in the full availability of a dedicated, local, not virtualized accelerator device. This is due to the need of the split-driver interleaved layers and (in the case of GPU remoting) the network infrastructure. In the best case the virtualization/remoting overhead is partially balanced by the improvements in computation performance. This happens for some application classes characterized by the need for high GPU calculations fed by a relative poor amount of input and output parameters. But, if we change the point of view from a strict performance oriented to a total costs of ownership perspective, the GPU remoting permits to build a cluster with a reduced number of GPUs. The use of GVirtuS middleware could be definitely effective if the applications are designed explicitly to take advantage from an hybrid architecture computation environment with a consistent costs reduction. The proposed evaluation tests have the target to demonstrate the effectiveness of the designed infrastructure rather than the mere performance that, as previously stated, is affected by many ineluctable components that could be mitigated with future technology improvements. It is possible to evaluate the overhead introduced by GVirtuS faced with the chance to run CUDA code or no CUDA enabled devices. Mainly, the bottleneck is the communication overhead due to the use of the TCP communicator. This results in poor performance, especially stressed out when the GPU remoting is done outside the local dedicated network where the overhead is acceptable.

5.1 GPGPU virtualization and remoting

In this section the results of CUDASW++ (test 1) and SRAD (test 2) to test performance for x86_64/x86_64/GPU in three different approaches have been showed.

Three test scenarios are presented:

- No virtualization: the CUDA code is executed using regular CUDA libraries. This is a measurement of the blank.
- Localhost remoting: the CUDA code is executed using a remoted CUDA device hosted on the same machine. This test verifies the effectiveness of GVirtuS libraries.
- Virtualization: the CUDA code is executed on a virtual machine hosted on the same physical host where the CUDA devices are connected to the PCIe bus.

Table 1 SRAD parameters

R	C	y1	y2	x1	x2	L	I
2048	2048	0	31	0	31	.5	10
2048	2048	0	31	0	31	.5	100
4096	4096	0	31	0	31	.5	10
4096	4096	0	31	0	31	.5	100

Table 2 SRAD performances

Size	Iterations	No Virtualization	Localhost Remoting	Virtualization
2048x2048	10	0.451s	0.626s	2.969s
2048x2048	100	1.119s	3.458s	27.872s
4096x4096	10	0.948s	2.005s	2.997s
4096x4096	100	3.781s	12.283s	27.946s

The Test 1 leverages on CUDASW++ version 2.0.11 executed with parameters -query P01008.fasta -db uniprot_sprot.fasta -use_single 0. The database used for the test is uniprot_sprot.fasta. This is the last release of the Swiss-Prot database released by UniProt, a scientific community with a comprehensive, high-quality and freely accessible resource of protein sequence and functional information. The database contains 550552 sequence entries [2]. The query used for the test is P01008.fasta. This is an example query sequence suggested by the CUDASW++ documentation.

The Test 2 leverages on benchmark provided by Rodinia SRAD casted with the parameters shown in Table 1: the first parameter, R, is the number of rows in the domain; the second parameter, C, is the number of columns in the domain. Currently, the GPU implementation of SRAD only supports a dimension of kernel that can be divided by 16. The kernel has square shape. The parameters from third to sixth represent respectively the y1,y2,x1,x2 positions of the speckle. The seventh parameter is the lambda value (L). The last parameter is I, the number of iterations.

The tests ensure the effectiveness of the GVirtuS framework because the results of the execution through CUDA and through GVirtuS coincide. As of the performances in the execution of CUDASW++, no significative overhead is introduced by the use of GVirtuS in the Localhost remoting scenario, while in the virtualization scenario the overhead introduced has to be correlated to the virtual environment. The execution of SRAD is impacted by the involvement of GVirtuS as shown in Table 2. The reason of this behaviour has to be found in the data intensive nature of this test so the bottleneck is represented by the TCP/IP communicator. Furthermore the TCP/IP communicator is not intended for performance purpose (Figure 8).

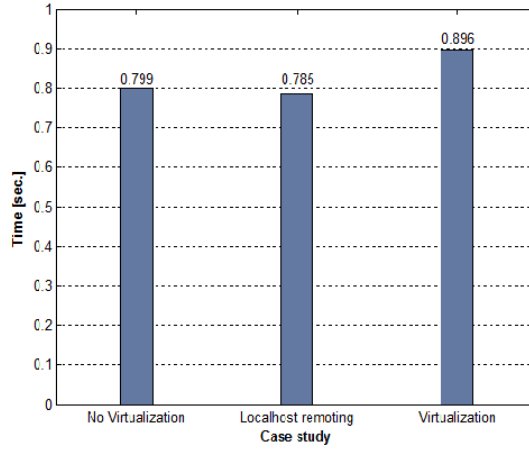


Fig. 8 Execution time of SWCUDA++ in the three different approach.

5.2 High-end ARM GPU cluster

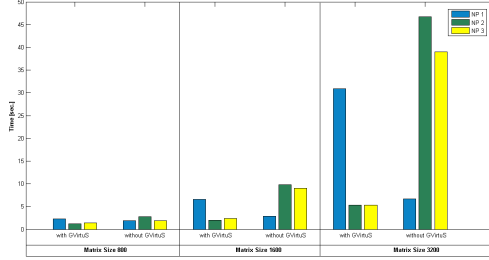
The results of MPI Matrix multiplication program showing the performance test results for a ARM/x86/GPU setup are presented in this section. In this experiment the MPI Matrix multiplication program has been used, in order to investigate about the behavior of GVirtuS in a scenario where a x86 machine is used as an accelerator node of a high-end ARM based cluster. In our setup each computing node is provided by an on-board K20A NVIDIA CUDA enabled GPU with 192 cores, while the accelerator node is powered by a couple of NVIDIA Titan X. This benchmark has been performed with two problem size: 1600x1600x800 and 3200x3200x1600. The experiment compares the performance of the on-board GPU and GVirtuS remoted on both problems size (Table 3). The ARM based cluster is built on 3 nodes each provided by 4 CPU cores. The MPI Matrix multiplication program uses MPI, but it is not OpenMP enabled, so runs were performed using up to 3 MPI computing processes.

Results demonstrate the use of GVirtuS remoted CUDA acceleration is convenient especially when the problem size increases: the weight of the latency due to the communication decreases, as expected. The overall performances are improved by the MPI parallel approach when the CUDA is used locally, but the limited amount of node memory and number of nodes prevented to investigate more in this direction.

When the number of MPI processes increases over 2, benchmarks are no more suitable for classic parallel programming efficiency and speedup analysis, but could be useful for some speculations about GVirtuS and its use in GPU remoting. When GVirtuS will fully support the multithreading, the use of the matrix multiplication enabled for both distributed and shared memory could provide a better performance test for this kind of applications (Figure 9).

Table 3 Matrix multiplication performances using internal and remotd GPU

Size Size	NP1	NP2	NP3	NP1 GVirtuS	NP2 GVirtuS	NP3 GVirtuS
800x800	2.812s	1.948s	1.895s	1.238s	1.383s	2.115s
1600x1600	9.813s	9.004s	5.846s	2.201s	2.411s	2.795s
3200x3200	46.754s	39.061s	30.388s	5.341s	5.280s	6.571s

**Fig. 9** Implementation of Matrix Multiplication multitask.

6 Related works

GPU virtualization solutions to GPGPU as GVirtuS have been implemented in research projects as rCUDA (Remote CUDA) [33] e DS-CUDA (Distributed-Shared CUDA) [17]. They all use an approach similar to GVirtuS, providing CUDA API wrappers on the front-end application in the guest OS while the back-end in the host OS accesses to the CUDA devices.

Table 4 shows the main differences on the CUDA toolkit supported, the implementation of various communicator components to connect the front-end and back-end, the re-compiling needed, the concurrent remote usage of CUDA devices in a transparent way, the support for x86 and ARM processors and, finally, the type of license.

- CUDA Toolkit supported: all GPGPU computing solutions mentioned implement the functions in the CUDA Runtime API, but the graphic relevant APIs, such as OpenGL and Direct3D interoperability, are not supported. A common restriction for GVirtuS and DS-CUDA is the asynchronous APIs implemented as aliases to their synchronous counterparts.
- Communicator: a communicator is a key piece because it connects the guest and host operating systems. One of the main differences lies in the use of the communication technique. In GVirtuS communicators are independent from hypervisor, virtualized technology and from the cooperation protocols between front-end and back-end. GVirtuS already provides several Communicator subclasses such as TCP/IP, Unix sockets, VMSocket (high performance communicator for KVM based virtualization), and VMCI (VMWare efficient and effective communication channel for VMWare based virtualization). By default, rCUDA and DS-CUDA use InfiniBand Verbs,

Table 4 CUDA virtualization features comparison table

GPU Vir.	RT	DRV	Comm	Rebuild	Plug-In	License
GVirtuS	6.5		TCP/IP, SHMem, ...		Yes	LGPL
rCUDA	5.5	Yes	IB, TCP/IP			Proprietary
DS-CUDA	4.5		IB, TCP/IP	Needed		GPL

and TCP sockets in case the network infrastructure does not support InfiniBand in the guest and host communication.

- Plug-in architecture: while GVirtuS is a general-purpose virtualization service with a plug-in architecture, which can load modules of CUDA and OpenCL and use different GPU devices, rCUDA and DS-CUDA allow to manage only NVIDIA GPUs. The main aim of GVirtuS is to provide a flexible tool capable to adapt itself to any possible scenario, GVirtuS competitors aim is just to provide NVIDIA support.
- Computing architecture: in the last years, the use of remote GPUs and low-power processors for acceleration of scientific applications has become an important case study. GVirtuS is a tool to virtualize heterogeneous architectures. It is based on a split-driver model independent of the computing architecture ARM and x86_64. DS-CUDA is going to use Android tablets and smartphones to run the executable CUDA file [24]. rCUDA carried experimental study on scientific applications with different hardware platforms [5].
- Transparency and re-compiling: the main goal of GVirtuS is to provide a fully transparent virtualization solution, that is CUDA enabled software has to be executed without any further modification of binaries and the source code of applications does not need to be modified in order to use remote GPUs. Transparency is an important common feature of the presented virtualization GPUs systems. DS-CUDA needed of a re-compiling in order to build an executable for the application program, this latter has a DS-CUDA preprocessor dscudacpp to handle CUDA C/C++ extensions.
- Licence: GVirtuS and DS-CUDA are open source projects, the former is licensed under the LGPL (Lesser General Public License), while the latter is licensed under the GPLv3 (General Public License version 3). The rCUDA technology is own by the Parallel Architectures Group from Universitat Politècnica de Valencia (Spain). The Software is distributed for free under specified terms and conditions of use.

7 Conclusions and future directions

In this paper were presented our results about the design and the implementation of an updated CUDA wrapper library for the GVirtuS framework in order to accelerate sub-clusters of inexpensive low power demanding ARM based boards. We used high-end GPGPU devices providing an experimental evaluation of the possibilities that state-of-the-art technology offers in nowadays

HPC facilities [31], as well as low-power alternatives offer for the acceleration of scientific applications using remote graphics processors.

The performed experiments demonstrate how convenient is the path we followed as trailblazer in the hunt for the next big thing in the off-the-shelf commodity high performance computing clusters. The latest GVirtuS release tested in a x86_64 virtualization and remoting performs enough to consider feasible the use of our approach in real world production applications, especially if enhanced with an Infiniband communicator component. This because with the availability of the needed hardware testbed, the communication plugin component will evolve in order to support the Infiniband network because it is expected that the higher bandwidth allows remote GPU virtualization frameworks to experience communication performances similar to the PCIe on the path between the local GPGPU and the remote GPU resource [32], [30]. Due to the unavailability of real world applications fitting the available ARM cluster, GVirtuS has been tested using an ad hoc distributed memory matrix multiplication software [14] and accelerated CUDA kernels working on local or x86 remoted high-end GPU device [18].

On short and medium term, we are working on the GVirtuS over all improvement in order to implement a production service for GPGPU computation offloading dedicated to high end server machines and mobile devices. A custom Java/Android friendly front-end implementation will enable to GPGPU computing the most part of low-power integrated systems and devices. The final destination of this research is provisioning a full production software environment for advanced earth system simulations and analysis based on science gateways, workflow engines and high performance cloud computing [27], giving a support for the next generation of scientific dissemination tools [29] and the smart city management in case of extreme weather events.

Acknowledgements This research has been supported mainly by the Grant Agreement number: 644312 - RAPID - H2020-ICT-2014/H2020-ICT-2014-1 "Heterogeneous Secure Multi-level Remote Acceleration Service for Low-Power Integrated Systems and Devices", in part by the project IZS ME04/12 RC/C78C120017001 "Mapping Escherichia Coli and Salmonella pollution in mussel farm areas and model prediction comparisons", in part by the University of Naples Parthenope - Department of Science and Technologies "Weather/marine extreme event simulation with Galaxy-ES (Earth System) scientific workflow engine and cloud computing tools" Research Project, and in part by the University of Naples Federico II - Department of Mathematics "Approcci Innovativi per la Risoluzione di Modelli di Interesse nelle Simulazioni Computazionali" Research Project Grant Agreement.

References

1. Armand F., M. Gien, G. Maign, and G. Mardinian, "Shared device driver model for virtualized mobile handsets.", Proceedings of the First Workshop on Virtualization in Mobile Computing, ACM, 2008, pp. 12-16.
2. A.M. Bairoch, R. Apweiler, C. H. Wu, W. C. Barker, B. Boeckmann, S. Ferro Rojas, E. Gasteiger et al. "The universal protein resource (UniProt)." Nucleic acids research 33, Database issue, 2005, pp. D154-9.
3. Bell N., and M. Garland, Efficient sparse matrix-vector multiplication on CUDA, NVIDIA Technical Report NVR-2008-004, Nvidia Corporation, 2008.

4. Caruso P. G. Laccetti and M. Lapegna, "A performance contract system in a grid enabling, component based programming environment" in *Advances in Grid Computing-EGC 2005*, LNCS vol. 3470, Springer Verlag, 2005, pp. 982-992.
5. Castello A., J. Duato, R. Mayo, A. J. Pena, E. S. Quintana-Ort, V. Roca, and F. Silla, "On the use of remote GPUs and low-power processors for the acceleration of scientific applications", in *The Fourth International Conference on Smart Grids, Green Communications and IT Energy-aware Technologies (ENERGY)*, 2014, pp. 57-62.
6. Dagum L. and R. Eon. "OpenMP: an industry standard API for shared-memory programming", *IEEE Computational Science and Engineering*, 5, 1, 1998, pp. 46-55.
7. Di Lauro R., F. Giannone, L. Ambrosio and R. Montella, "Virtualizing general purpose GPUs for high performance cloud computing: an application to a fluid simulator", in *IEEE 10th International Symposium on Proc. of Parallel and Distributed Processing with Applications (ISPA)*, 2012, pp. 863-864.
8. Di Lauro R., F. Lucarelli, and R. Montella. "SaaS-sensing instrument as a service using cloud computing to turn physical instrument into ubiquitous service." In *2012 IEEE 10th International Symposium on Parallel and Distributed Processing with Applications*, pp. 861-862. IEEE, 2012.
9. Foster I., Y. Zhao, I. Raicu and S. Lu. "Cloud computing and grid computing 360-degree compared", in *IEEE Grid Computing Environments Workshop GCE 08*, 2008, pp. 1-10.
10. Giunta G., P. Mariani, R. Montella, and A. Riccio. "pPOM: A nested, scalable, parallel and Fortran 90 implementation of the Princeton Ocean Model." *Environmental Modelling & Software* 22, no. 1 (2007): 117-122.
11. Garland M., S. Le Grand, J. Nickolls, J. Anderson, J. Hardwick, S. Morton, E. Phillips, Y. Zhang and V. Volkov, "Parallel computing experiences with CUDA." *IEEE Micro*, 28, 4, 2008, pp. 13-27.
12. Giunta G., R. Montella, G. Agrillo and G. Coviello, "A GPGPU transparent virtualization component for high performance computing clouds", in *EuroPar 2010 Parallel Processing*, LNCS vol. 6271, 2 Springer Verlag, 2010, pp. 379-391.
13. Giunta G., R. Montella, G. Laccetti, F. Isaila, and F. Blas. "A GPU accelerated high performance cloud computing infrastructure for grid computing based virtual environmental laboratory." *Advances in Grid Computing* (2011): 35-43.
14. Gropp W., "MPICH2: A new start for MPI implementations", in *Recent Advances in Parallel Virtual Machine and Message Passing Interface 2002*, LNCS vol. 2474, Springer, 2002, pp. 7.
15. Gupta V., A. Gavrilovska, K. Schwan, H. Kharche, N. Tolia, V. Talwar, and P. Ranganathan. "GVim: GPU-accelerated virtual machines." In *Proceedings of the 3rd ACM Workshop on System-level Virtualization for High Performance Computing*, ACM, 2009, pp. 17-24.
16. Herrera A. "NVIDIA GRID: Graphics Accelerated VDI with the Visual Performance of a Workstation." Nvidia Corp, 2014.
17. Kawai A., K. Yasuoka, K. Yoshikawa and T. Narumi. "Distributed-shared CUDA: Virtualization of large-scale GPU systems for programmability and reliability.", 2012.
18. Karunadasa, N. P. and D. N. Ranasinghe. "Accelerating high performance applications with CUDA and MPI." In *Industrial and Information Systems (ICIIS), 2009 International Conference on*, IEEE, 2009, pp. 331-336.
19. Kehne J., J. Metter and F. Bellosa. "GPUswap: Enabling Oversubscription of GPU Memory through Transparent Swapping." In *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ACM, 2015, pp. 65-77.
20. Laccetti G., R. Montella, C. Palmieri and V. Pelliccia. "The High Performance Internet of Things: Using GVirtuS to Share High-End GPUs with ARM Based Cluster Computing Nodes." In *Parallel processing and Applied Mathematics 2013*, LNCS vol. 8384, Springer Verlag Berlin Heidelberg, 2013, pp. 734-744.
21. Ligowski L. and W. Rudnicki. "An efficient implementation of Smith Waterman algorithm on GPU using CUDA, for massively parallel scanning of sequence databases." In *Parallel & Distributed Processing, 2009. IPDPS 2009. IEEE International Symposium on*, IEEE, 2009, pp. 1-8.

22. Liu Y., B. Schmidt and D. L. Maskell. "CUDASW++ 2.0: enhanced Smith-Waterman protein database search on CUDA-enabled GPUs based on SIMT and virtualized SIMD abstractions." *BMC research notes* 3, no. 1, 2010, p. 93.
23. Manavski S. A., and G. Valle. "CUDA compatible GPU cards as efficient hardware accelerators for Smith-Waterman sequence alignment." *BMC bioinformatics* 9, no. 2, 2008, p. 1.
24. Martinez-Noriega E.J., E. Josafat, A. Kawai, K. Yoshikawa, K. Yasuoka and T. Narumi. "CUDA Enabled for Android Tablets through DS-CUDA.", 2013.
25. Montella R. and I. Foster. "Using hybrid grid/cloud computing technologies for environmental data elastic storage, processing, and provisioning." In *Handbook of Cloud Computing*, Springer US, 2010, pp. 595-618.
26. Montella R., G. Giunta and G. Laccetti. "Virtualizing high-end GPGPUs on ARM clusters for the next generation of high performance cloud computing." *Cluster computing* 17, no. 1, 2014, pp. 139-152.
27. Montella R., D. Kelly, W. Xiong, A. Brizius, J. Elliott, R. Madduri, K. Maheshwari et al. "FACEIT: A science gateway for food security research." *Concurrency and Computation: Practice and Experience* 27, no. 16, 2015, pp. 4423-4436.
28. Montella R., G. Giunta, G. Laccetti, M. Lapegna, C. Palmieri, C. Ferraro and V. Pelliccia. "Virtualizing CUDA enabled GPGPUs on ARM clusters." In *Parallel Processing in and Applied Mathematics 2015*, LNCS vol. 9574, Springer Verlag Berlin Heidelberg, 2016.
29. Pham Q., T. Malik, I. Foster, R. Di Lauro and R. Montella. "SOLE: linking research papers with science objects." In *Provenance and Annotation of Data and Processes 2012*, LNCS vol. 7525, Springer Verlag Berlin Heidelberg, 2012, pp. 203-208.
30. Prades J., C. Reao and F. Silla. "CUDA acceleration for Xen virtual machines in infiniband clusters with rCUDA." In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ACM, 2016, p. 35.
31. Rajovic N., A. Rico, N. Puzovic, C. Adeniyi-Jones and A. Ramirez. "Tibidabo: Making the case for an ARM-based HPC system." *Future Generation Computer Systems* 36, 2014, pp.322-334.
32. Reao C., R. Mayo, E.S. Quintana-Orti, F. Silla, J. Duato, A.J. Pea, Influence of InfiniBand FDR on the performance of remote GPU virtualization, in proceedings of the 2013 IEEE International Conference on Cluster Computing, Indianapolis, USA, October 2013.
33. Reao C., F. Silla, A. J. Pena, G. Shainer, S. Schultz, A. Castello, E. S. Quintana-Orti and J. Duato. "POSTER: Boosting the performance of remote GPU virtualization using InfiniBand connect-IB and PCIe 3.0." In *Cluster Computing (CLUSTER)*, 2014 IEEE International Conference on, IEEE, 2014, pp. 266-267.
34. Shi L., H. Chen, J. Sun and K. Li. "vCUDA: GPU-accelerated high-performance computing in virtual machines." *Computers, IEEE Transactions on* 61, no. 6, 2012, pp. 804-816.
35. Shuai C., M. Boyer, J.Meng, D. Tarjan, J. W. Sheaffer, and K. Skadron, "A performance study of general-purpose applications on graphics processors using CUDA", *Journal of Parallel and Distributed Computing*, 68, 10, 2008, pp. 1370-1380.
36. Shuai C., M. Boyer, J.Meng, D. Tarjan, J. W. Sheaffer, Sang-Ha Lee, and K. Skadron "Rodinia: A benchmark suite for heterogeneous computing" , *Proc. of the IEEE International Symposium on Workload Characterization - IISWC 2009*, 2009, pp. 44-54.
37. Sourouri M., T. Gillberg, S. B. Baden and X. Cai. "Effective multi-GPU communication using multiple CUDA streams and threads." In *Parallel and Distributed Systems (ICPADS)*, 2014 20th IEEE International Conference on, IEEE, 2014, pp. 981-986.
38. Szafaryn L. G. , K. Skadron and J. J. Saucerman. "Experiences Accelerating MATLAB Systems Biology Applications." In *Proceedings of the Workshop on Biomedicine in Computing: Systems, Architectures, and Circuits (BiC) 2009*, in conjunction with the 36th IEEE/ACM International Symposium on Computer Architecture (ISCA), June 2009.
39. Volkov V. and J. W. Demmel. "Benchmarking GPUs to tune dense linear algebra." In *High Performance Computing, Networking, Storage and Analysis*, 2008. SC 2008. International Conference for, IEEE, 2008, pp. 1-11.
40. Yang C., C. Huang and C. Lin. "Hybrid CUDA, OpenMP, and MPI parallel programming on multicore GPU clusters." *Computer Physics Communications* 182, no. 1, 2011, pp. 266-269.